

Switching on the parts of the app that are waiting

Several screens in your tracker look empty or broken right now. **None of that is a fault — the app is finished and has nowhere to put its information yet.** Today you make that somewhere.

Four files. Each one goes in **twice**. Every keystroke is written out on pages 5 to 8 — one page per file, ten numbered steps each. Nothing is left for you to work out.

1	2	3	4
0026	0027	0028	0029
The list of stages an organization can be at	Goals, and the counting behind the badges	Where the Jira settings will live	Organization IDs and their branch lists

Two things that make this safer than it sounds

Each file is all-or-nothing. If anything goes wrong partway, the file undoes itself completely and nothing is left behind. There is no such thing as half-applied here.

Each file checks its own work. It makes some pretend data, tests that the rules behave, deletes the pretend data, then reports in plain sentences what it proved. Those sentences are the grey lines you will be counting.

The instruction that matters most

Every file goes in twice — eight runs in total, not four. Page 5 shows exactly how, and it is easier than it sounds: the second run needs no trip back to Terminal.



The words you will see

Six of them. Come back here whenever one turns up and looks alarming.

Table	One kind of thing in the database, like one sheet in a spreadsheet. Today you are adding several new empty ones that the app already knows how to fill.
SQL Editor	A page on the Supabase website with a big empty box and a Run button. You paste into the box and press Run. That is the whole tool.
NOTICE a grey line	A progress message: “here is what I just did and checked”. These are good. You will be counting them — 4, 4, 3, 5.
ERROR a red line	Something refused to happen and the file undid itself. This is the only word on screen that means stop. It always comes with a sentence saying what was wrong.
Probe inside the grey lines	The files call their self-tests “probes”. Probe 1 just means self-test 1 .
Rolled back ends a grey line	“The pretend data I made for that test has been deleted again.” The self-test tidying up after itself. Seeing it is a good sign.

The whole job in one sentence

Copy a file, paste it, press Run, count the grey lines, check none is red — then do that same file a second time. Four times over.



Before you start

What to have ready, and two things not to do.

- 1 **Your Supabase sign-in**, and about twenty minutes. Each file's two runs should be done together, but you can stop between files.
- 2 **A Terminal window**. Press `⌘ Space`, type **Terminal**, press `Enter`. Leave it open — you will come back to it four times.
- 3 **Nobody needs signing out**, nothing needs backing up, and the app keeps working normally the whole time.

Do not rename any stage names until all eight runs are done

Afterwards you will rename the seven stages into your programme's real words. **Wait until the very end**. The file recognises the stages by name, so renaming first makes it think they are missing and add all seven originals back alongside yours. Page 10 says more.

Do not archive the track called UHR

Nine of your ten tracks are already archived. `UHR` is the only live one, and the self-tests need a live track to hang their pretend data on. With none, all four files refuse to run at all.

Two things already checked and correct

The earlier files these four depend on are all in place, and your **Director** role is set up exactly as the self-tests need. So you should not see the word **SKIPPED** anywhere.

Open

0026

0027

0028

0029

Check

1

Open the right page

Do this once. Then leave the tab open for the whole job.

1

Go to **supabase.com** and sign in.

2

Open your project, click **SQL Editor** in the left menu, then **New query**.

A large empty box with a **Run** button near it.

3

Check the address bar contains `lrysgpbkmuqgzsjesfkr` before you paste anything.

`lrysgpbkmuqgzsjesfkr`

SQL Editor

+ New query

Run ⌘↵

-- this is the big box. Everything you paste goes here.

If you have more than one Supabase project

It is genuinely easy to be looking at the wrong one. Everything would appear to work perfectly and none of it would have happened to your tracker. **Check the letters in the address bar, not the project's display name.**

Pasting is not running

This page will hold pasted text forever and quietly save it as a draft. Paste and navigate away and **nothing has happened**, with no warning of any kind. ⌘↵ or the **Run** button is what actually does it.

Open

0026

0027

0028

0029

Check

2

File 1 of 4 — 0026

The stage ladder. Ten steps; steps 7 to 9 are the second run.

1

In **Terminal**, paste these two lines and press **Enter**:

```
cd /Users/aziz/Claude/nphiescore  
pbcopy < supabase/migrations/0026_map_node_stages.sql
```

Nothing appears to happen. **That is success** — the file is now on your clipboard, replacing the two lines you just copied.

2

Switch to your browser, to the SQL Editor tab from page 4.

3

Click inside the big box, then press **⌘ A** and **Delete**.

The box is empty.

4

Press **⌘ V**.

The box fills with thousands of lines. Do not read them.

5

Press **⌘ ↵**. Wait a few seconds.

6

Count the grey lines underneath.

4 Four grey lines, each beginning NphiesCore 0026 probe. **No red line.**

7

Now the second run. Press **⌘ A**, **Delete**, then **⌘ V** again.

The same file goes back in — it is still on your clipboard, so you do **not** go back to Terminal.

8

Press **⌘ ↵** again.

9

Count again: four again, saying the same things as the first time.

10

File 1 done. Turn the page.

Open

0026

0027

0028

0029

Check

3

File 2 of 4 — 0027

Goals, and the numbers on the badges. Same ten steps.

1

In **Terminal**, paste these two lines and press **Enter**:

```
cd /Users/aziz/Claude/nphiescore
pbcopy < supabase/migrations/0027_map_node_goals_and_counts.sql
```

Nothing appears to happen. Correct.

2

Switch to the browser, SQL Editor tab.

3

Click in the big box, press **⌘ A** then **Delete**.

Empty — the previous file is cleared out.

4

Press **⌘ V**.

5

Press **⌘ ↵**.

6

Count the grey lines.

4 Four again, this time beginning NphiesCore 0027 probe. No red line.

7

Second run: **⌘ A**, **Delete**, **⌘ V**.

8

Press **⌘ ↵** again.

9

Four again, saying the same things.

10

File 2 done. Turn the page.

This file must come after file 1 — and it checks

Goals can point at a stage, so it needs the stage ladder to exist. Run it first and it refuses with a clear sentence rather than doing anything strange.

Open

0026

0027

0028

0029

Check

4

File 3 of 4 — 0028

Where the Jira settings will live. **Note the count changes to three.**

1

In Terminal, paste and press **Enter**:

```
cd /Users/aziz/Claude/nphiescore  
pbcopy < supabase/migrations/0028_jira_settings.sql
```

2

Switch to the browser, SQL Editor tab.

3

Click in the big box, **⌘ A**, **Delete**.

4

Press **⌘ V**.

5

Press **⌘ ↵**.

6

Count the grey lines.

3

Three this time, not four. That is expected — this file has three self-tests. No red line.

7

Second run: **⌘ A**, **Delete**, **⌘ V**.

8

Press **⌘ ↵** again.

9

Three again, saying the same things.

10

File 3 done. Turn the page.

Jira stays switched off and invisible

This file creates the place settings would go and puts **nothing** in it. Jira does not appear in the app until you deliberately set it up, which is a separate optional job. Running this commits you to nothing.

Open

0026

0027

0028

0029

Check

5

File 4 of 4 — 0029

Organization IDs and branch lists. **Five grey lines this time.**

1

In Terminal, paste and press **Enter**:

```
cd /Users/aziz/Claude/nphiescore  
pbcopy < supabase/migrations/0029_org_identity.sql
```

2

Switch to the browser, SQL Editor tab.

3

Click in the big box, **⌘ A**, **Delete**.

4

Press **⌘ V**.

5

Press **⌘ ↵**.

6

Count the grey lines.

5

Five — the most of any file. No red line.

7

Second run: **⌘ A**, **Delete**, **⌘ V**.

8

Press **⌘ ↵** again.

9

Five again, saying the same things.

10

All four files done — that is eight runs. Turn the page and check it worked.

What this last file is for

A hospital group is one organization in your tracker, but NPHIES gives it **several facility IDs** — Hafar Al-Batin has seven — and one certificate covers all of them. There was nowhere to keep that list, or the official Organization ID. This makes both places. It adds nothing to your screens today.

Open

0026

0027

0028

0029

Check

6

Check it worked

Two more things to paste. Neither changes anything — they only look.

1

In the big box:  **A**, **Delete**. Then type or paste this and press  .

```
select relname from pg_class
where relname in ('map_node_stages', 'map_node_progress',
                  'map_node_goals', 'jira_settings', 'map_node_branches')
order by 1;
```

5

Five rows means it all landed. Four rows means only the last file is missing — go back to page 8 and do it again. Zero rows means nothing ran at all: check the address bar, and re-read page 4.

2

The safety check. It is long, so it has the next page to itself. **Turn to page 10**, copy the whole block, paste it into the big box the same way, and press  .

One row back, with many columns. Only three of them matter.

Why the second check exists

None of the four files touches anything that already existed — they only add new, empty things. So those three values **cannot** have changed. If they have, something else was run at some point, and it is far better to know now than in three months.

Open

0026

0027

0028

0029

Check

The safety check, in full

Copy this whole block into the big box and run it. It reads only — it changes nothing.

```
select
(select count(*) from public.track_groups) as g_0018,
(select count(*) from information_schema.columns
 where table_schema='public' and table_name='tracks' and column_name='group_id') as c_0018,
(select count(*) from pg_proc p join pg_namespace n on n.oid=p.pronamespace
 where n.nspname='public' and p.proname='nudge_entry') as f_0019,
(select count(*) from information_schema.tables
 where table_schema='public' and table_name='ai_usage') as t_0020,
(select count(*) from information_schema.columns
 where table_schema='public' and table_name='notification_prefs'
 and column_name='ai_enabled') as c_0021,
(pg_get_functiondef('public.entries_guard_insert()::regprocedure')
 like '%nudged_by := null%') as f_0022,
(select pg_get_constraintdef(oid) from pg_constraint
 where conname='claim_counters_scope_ck') as scopes,
(select count(*) from public.map_node_kinds) as k_0023,
(select count(*) from pg_trigger t join pg_class c on c.oid=t.tgrelid
 where c.relname='map_nodes' and t.tgname='map_nodes_tree_ck_trg'
 and t.tgdeferrable and t.tginitdeferred) as d_0023,
(select pg_get_indexdef(i.indexrelid) ilike '%nulls not distinct%'
 from pg_index i join pg_class c on c.oid=i.indexrelid
 where c.relname='map_nodes_sibling_name_uidx') as n_0023,
(select count(*) from public.use_cases) as u_0024,
(select count(*) from information_schema.tables
 where table_schema='public' and table_name='map_node_use_cases') as t_0024,
(select count(*) from information_schema.columns
 where table_schema='public' and table_name='entries' and column_name='node_id') as c_0024,
(select count(*) from pg_trigger
 where tgrelid='public.entries::regclass' and tgname='entries_map_sync') as g_0024,
(select count(*) from public.roles) as r_0025,
(select count(*) from public.role_permissions where granted) as p_0025,
(select count(*) from public.profiles where role_id is null) as n_0025,
(pg_get_functiondef('public.is_admin()::regprocedure') like '%has_perm%') as f_0025,
(select count(*) from information_schema.columns
 where table_schema='public' and table_name='profiles' and column_name='position') as c_0025,
-- the amendment, both directions. The first must be true and the second false;
-- `w_0025 = false` means the Director role grants nothing, and `x_0025 = true`
-- means a Director can grant themselves workspace.admin,
(select count(*) from pg_policies
 where schemaname='public' and tablename='map_nodes' and policyname='map_nodes_insert'
 and coalesce(with_check,'') like '%structure.edit%') as w_0025,
(select count(*) from pg_policies
 where schemaname='public' and tablename='role_permissions'
 and policyname='role_permissions_update'
 and (coalesce(qual,'')||coalesce(with_check,'')) like '%structure.edit%') as x_0025;
```



Of all the columns that come back, read these three: **x_0025** must be **0** · **f_0025** must be **true** · **w_0025** must be **1**.

Read x_0025 first. If it is ever 1, stop and tell me straight away — it would mean anyone holding Director could give themselves full control.

Ignore every other column — and where this came from

The rest are a fingerprint of everything set up before today; I compare them if anything ever looks odd. You only ever need the three above.

Printed here on 21 August 2026 from [docs/PENDING-MIGRATIONS.md](#) so this guide is self-contained. If the two ever disagree, **the file in the repository is the real one.**

[Open](#)
[0026](#)
[0027](#)
[0028](#)
[0029](#)
[Check](#)

Three things that look like failures and are not

A line containing SKIPPED

One half of a self-test could not run here. **Everything was still installed correctly** and the other half ran in full. Given how your roles are set up you should not see this — but if you do, mention it to me and carry on.

A line saying the ladder has been edited

The stage names are no longer the originals, so you have renamed them. Correct and expected if you have got that far. Not an error.

The green Success. No rows returned

This is what success looks like for these files. They are not meant to return a table — their output is the grey lines above it. **“No rows returned” is good news.**

What a real problem looks like

```
ERROR: NphiesCore 0026 FAILED: <a sentence saying what is wrong>
```

Red, and beginning with **ERROR**. When this happens **nothing at all was changed** — the file undid itself completely. Send me the sentence; fixing it and running again costs nothing.

The one mistake nothing can catch for you

Renaming the seven stages before all eight runs are done. The file finds the stages by name. Rename them to your own words first and it will not recognise them, will assume they are missing, and will add all seven English originals back — leaving fourteen stages, seven of which nobody wants, in every menu.

Fixable by deleting the extras, but tedious. **Finish all eight runs, then rename.**

Tell me, and stop there

Send me a message saying “SQL is done”. Your next piece of work is looking at the app, not more of this.

TICK THESE OFF

- ☐ **0026** run **twice** — 4 grey lines each time
- ☐ **0027** run **twice** — 4 grey lines each time
- ☐ **0028** run **twice** — 3 grey lines each time
- ☐ **0029** run **twice** — 5 grey lines each time
- ☐ The first check returned **5 rows**
- ☐ The safety check read `0`, `true`, `1`
- ☐ No stage has been renamed yet

What I do next

First I take the old practice data off — twenty-two made-up organizations from an earlier trial, which have to come off **before** anything new goes on, because they share a name with the new set. Then I load a small realistic demo so you have something to look at, and check everything from my side.

Then the interesting part is yours

You open the app and the Portfolio view is no longer empty. You rename the seven stages into your programme's real words, add the Arabic for each — deliberately left blank, because a guessed translation looks exactly like a reviewed one on screen — and decide how many days on a stage counts as “stuck”. **Until you set that last one the Stalled view stays empty on purpose**, because a threshold nobody chose is a number the app would start chasing people with.